CENG 111 Algoritmalar ve Programlama 3: C Programlamaya Giriş

Öğr.Gör. Şevket Umut ÇAKIR

Pamukkale Üniversitesi

Hafta 3

Anahat

1. Basit C Programı
2. Toplam Programı
3. C'de Aritmetik
4. Eşitlik ve İlişkisel Operatörler

Basit C Programı

```c
// kod2_1.c
// İlk C programı
#include <stdio.h>

// main fonksiyonu programın başlangıç noktasıdır
int main(void) {
    printf("C'ye hoşgeldiniz!\n");
} // main fonksiyonunun sonu
```

Çıktı:

```
C'ye hoşgeldiniz!
```

Basit C Programı

Açıklama/Yorum Satırları(Comments)

```
1 // kod2_1.c
2 // İlk C programı
```

- // ile başlayan satırlar açıklama satırlarıdır
- Kodu belgelendirmek için açıklamalar eklenir
- Programlar çalışırken açıklamaların çalışmaya etkisi olmaz, işlem yapılmaz
- /* ... */ ifadesi içinde ile çok satırlı açıklamalar eklenebilir
  - */ kapatma ifadesini unutmak hataya neden olur

Basit C Programı

#include Önişlemci Direktifi(Preprocessor Directive)

```
3 #include <stdio.h>
```

- Üstteki ifade bir önişlemci direktifidir
- Önişlemci, derleme öncesi # ile başlayan satırları ele alır
- 3. satır, önişlemciye standart girdi/çıktı(standard input/output) başlığının(header) içeriğini koda dahil etmeyi sağlar
- 7. satırdaki `printf` komutu bu önişlemci direktifi ile koda dahil edilir
- Programın daha kolay okunması için boş satırlar ve boşluk karakterleri eklenebilir
  - Bunlar **beyaz boşluk(white space)** olarak bilinir ve derleyici tarafından göz ardı edilir

Basit C Programı

`main` Fonksiyonu

```
6 int main(void) {
```

- `main` her C programının bir parçasıdır
- `main`'den sonraki parantezler, bir fonksiyon bloğu olduğunu gösterir
- Her C programı çalışmaya `main` fonksiyonundan başlar
- Fonksiyonlar geri değer döndürebilir
  - `main` fonksiyonunun önünde bulunan `int` ifadesi, fonksiyonun geriye bir tamsayı döndürdüğünü gösterir
- Fonksiyonlar çalıştırıldığında dışardan bilgi alabilir
  - Parantez içindeki `void` ifadesi `main` fonksiyonunun herhangi bir bilgi/parametre almadığını gösterir
- Süslü parantez açılışı(left brace) {, fonksiyonun **gövdesini** başlatır, } ile de **gövde** sonlanır

Basit C Programı

Çıktı Komutu

```
7     printf("C'ye hoşgeldiniz!\n");
```

- ifadesi, ekrana bir **metin(string)** yazılmasını sağlar
  - `string` ifadeleri iki adet çift tırnak(") sembolü içinde yer alır
  - Tüm satır; `printf` fonksiyonuna çağrı(call), parantez içindeki argüman(argument) ve sondaki ; sembolü **komut(statement)** olarak adlandırılır
  - Sondaki f, biçimlendirilmiş(formatted) ifadesinin kısaltılmışıdır
- 7. satır çalışınca ekranda "C'ye hoşgeldiniz!" mesajı gösterilir

Kaçış Dizileri(Escape Sequences)

- Bir string içindeki (\) sembolü kaçış(escape) karakteridir
- `printf`'in olağan dışı bir şey yapması gerektiğini gösterir
- \ ve sonraki karakter bir **kaçış dizisi(escape sequence)** oluşturur

Kaçış dizisi	**Açıklama**
\n	İmleci bir sonraki satırın başına hareket ettirir
\t	İmleci bir sonraki yatay sekme durma noktasına hareket ettirir
\a	Mevcut imleç konumunu değiştirmeden sesli veya görsel bir uyarı üretir.
\\	Ters eğik çizginin(back slash) bir dizgede(string) özel bir anlamı olduğundan, \\ bir dizgeye ters eğik çizgi karakteri eklemek için gereklidir.
\"	Dizgeler(string) çift tırnak işareti içine alındığından, bir dizgeye çift tırnak işareti eklemek için \" gereklidir.

Bağlayıcı(Linker) ve Yürütülebilir Dosyalar

- `printf` ve `scanf` gibi fonksiyonlar standart kütüphane fonksiyonları C'nin bir parçası değildir
  - Örneğin derleyici `printf` veya `scanf`'deki yazım hatasını bulamaz
- `printf` komutu derlenirken, derleyici kütüphane fonksiyonuna bir "çağrı(call)" içerir
  - Derleyici fonksiyonun nerede olduğunu bilmez ama bağlayıcı(linker) bilir
- Bağlayıcı çalıştığında, kütüphane fonksiyonlarını bulur ve nesne programına uygun çağrıları ekler
- Bu aşamadan sonra program çalıştırılabilir dosya(executable) olarak adlandırılır
- Eğer fonksiyon adı yanlış yazılmışsa, bağlayıcı(linker) bunu algılar

Girinti Kuralları

- Girintiler kodun daha okunabilir olmasını sağlar
- Derleyici açısından bir önemi yoktur
- Genellikle bloklar içindeki kodlar girinti ile gösterilir
  - Örn: `main` fonksiyonu gövdesi
- Girinti için kullanılan kural, kodun her konumunda ortak olmalıdır
  - Sekme(tab) karakteri
  - 4/8 boşluk karakteri

Çoklu printf

```c
// kod2_2.c
// İki printf ifadesi ile tek satır yazmak
#include <stdio.h>

// main fonksiyonu programın başlangıç noktasıdır
int main(void) {
    printf("C'ye ");
    printf("hoşgeldiniz!\n");
} // main fonksiyonunun sonu
```

Çıktı:

```
C'ye hoşgeldiniz!
```

Tek printf, çoklu satır

```c
// kod2_3.c
// İlk C programı
#include <stdio.h>

// main fonksiyonu programın başlangıç noktasıdır
int main(void) {
    printf("C'ye \nhoş\ngeldiniz!\n");
} // main fonksiyonunun sonu
```

Çıktı:

```
C'ye
hoş
geldiniz!
```

İki sayıyı toplayan program

```c
// kod2_4.c
// Toplama programı
#include <stdio.h>

// main fonksiyonu programın başlangıç noktasıdır
int main(void) {
    int tamsayi1 = 0; // Kullanıcının girdiği birinci değeri tutar
    int tamsayi2 = 0; // Kullanıcının girdiği birinci değeri tutar

    printf("İlk tamsayıyı girin: "); // prompt
    scanf("%d", &tamsayi1); // Bir tamsayı oku

    printf("İkinci tamsayıyı girin: "); // prompt
    scanf("%d", &tamsayi2); // Bir tamsayı oku

    int toplam = 0; // toplamı tutacak değişken
    toplam = tamsayi1 + tamsayi2; // toplam değerini toplam değişkenine ata

    printf("Toplam: %d\n", toplam); // toplamı yazdır
} // main fonksiyonunun sonu
```

İki sayıyı toplayan program

```
İlk tamsayıyı girin: 45
İkinci tamsayıyı girin: 72
Toplam: 117
```

- Değişkenler ve değişken tanımlama(definition)

```
7     int tamsayi1 = 0; // Kullanıcının girdiği birinci değeri tutar
8     int tamsayi2 = 0; // Kullanıcının girdiği birinci değeri tutar
```

- `tamsayi1` ve `tamsayi2` **değişkendir** ve türleri **tamsayıdır**
  - Değişkenler, programın daha sonra kullanmak üzere değer saklayabildiği bellekteki konumlardır
- Üstteki satırlar değişkenlere ilk değerlerini 0 olarak verirler(initialize)
  - Her değişkene başlangıç değeri vermek zorunlu olmasa da, hataları önlemeye yardımcı olacaktır

İki sayıyı toplayan program

- **Kullanmadan önce tanımlama kuralı**: C'de değişkenlerin kullanılmadan önce tanımlanması gerekmektedir
- Değişken adı herhangi bir geçerli tanımlayıcı olabilir.
  - Harfleri, rakamları ve _ sembolünü içerebilir
  - Rakamla başlayamaz
- C büyük/küçük harfe duyarlıdır, bu nedenle `a1` ve `A1` farklı tanımlayıcılardır
- Anlamlı değişken isimleri seçmek, bir programın kendi kendini belgelemesine yardımcı olur, bu nedenle daha az yorum gerekir
- Derleyici tanımlı tanımlayıcılar ile karışmaması açısından değişken isimlerini _ ile başlatmaktan kaçınmak gerekir
- Çok sözcüklü değişken isimleri programları daha okunur kılar
  - **snake_case**: Kelimeler arasına _ sembolü yerleştirilir(`toplam_komisyon`)
  - **camelCase**: İlk kelime küçük, diğer kelimelerin ilk harfleri büyük(`toplamKomisyon`)

İki sayıyı toplayan program

İstem(prompt)

```
10      printf("İlk tamsayıyı girin: "); // prompt
```

- Bu mesaj **istem(prompt)** olarak adlandırılır çünkü kullanıcıya spesifik bir eylemde bulunmasını söyler

İki sayıyı toplayan program

scanf

```
11      scanf("%d", &tamsayi1); // Bir tamsayı oku
```

- Üstteki ifade, kullanıcıdan bir değer almak için `scanf` kullanır(f: formatted)
- İki argümanı vardır:
  - `"%d"`: format kontrol dizgesi(decimal integer, onluk tamsayı)
  - `&tamsayi1`: adres operatörü ve değişken ismi
- `&` kullanımı genellikle acemi programcılar tarafından yadırganır veya kafa karıştırıcı bulunur
  - `&` sembolünü unutmak genellikle çalışma zamanında hata olarak karşımıza çıkar

İki sayıyı toplayan program

Atama

```
13      printf("İkinci tamsayıyı girin: "); // prompt
14      scanf("%d", &tamsayi2); // Bir tamsayı oku
15
16      int toplam = 0; // toplamı tutacak değişken
17      toplam = tamsayi1 + tamsayi2; // toplam değerini toplam değişkenine ata
```

- İfade "`toplam`, `tamsayi1 + tamsayi2` ifadesinin sonucunu alır" şeklinde okunur
- = ve +, ikili(binary) operatör olarak adlandırılır
  - Bir operatör ve 2 adet işlenenden(operand) oluşur

İki sayıyı toplayan program

Biçimli(formatlı) yazdırma

```
19      printf("Toplam: %d\n", toplam); // toplamı yazdır
```

- `printf` ile biçimli yazdırılır
- İki argümanı vardır
  - `"Toplam: %d\n"`: `"Toplam: "` metninden sonra bir tamsayı ve yeni satır karakteri geleceğini gösterir
  - `toplam`: tamsayının değeri verilir
- Aşağıdaki şekilde de yazılabilir

```
printf("Toplam: ", tamsayi1 + tamsayi2);
```

Bellek Kavramları

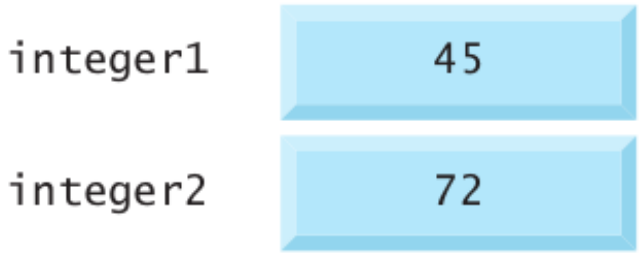
- Her değişkenin adı, türü, değeri ve konumu vardır
- Aşağıdaki işlem, kullanıcıdan bir sayı alır ve bu değeri `tamsayı1`'in bellek konumuna yerleştirir

```
11      scanf("%d", &tamsayi1); // Bir tamsayı oku
```



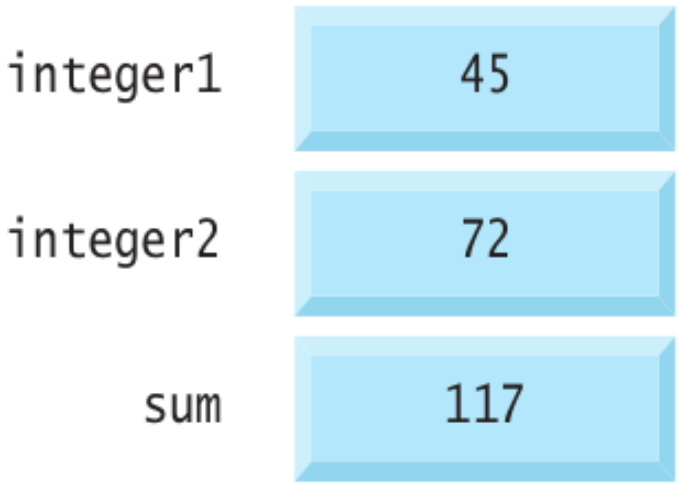
Bellek Kavramları

```
14      scanf("%d", &tamsayi2); // Bir tamsayı oku
```



Bellek Kavramları

```
17      toplam = tamsayi1 + tamsayi2; // toplam değerini toplam değişkenine ata
```



C'de Aritmetik

C işlemi	Aritmetik operatör	Cebirsel ifade	C ifadesi
Toplama	$+$	$f + 7$	`f + 7`
Çıkarma	$-$	$p - c$	`p - c`
Çarpma	$*$	bm, $b \cdot m$	`b * m`
Bölme	$/$	x/y, $\frac{x}{y}$	`x / y`
Kalan	$\%$	r mod s	`r % s`

- Bir tamsayıyı başka bir tamsayıya bölmek sonucunda değer yine tamsayı olur
- Sıfıra bölme tanımsızdır ve hata sebebidir
- İşlemleri gruplamak için parantezler kullanılabilir

İşlem Önceliği

1. Parantez içinde gruplanmış ifadeler ilk değerlendirilir
   - ((a + b) + c)
2. *, / ve % işlemleri sonra yapılır
3. + ve - işlemleri
4. Atama (=) işlemi

Cebirsel İfadelerin C karşılığı

- Cebir: $m = \frac{a+b+c+d+e}{5}$

Cebirsel İfadelerin C karşılığı

- Cebir: $m = \frac{a+b+c+d+e}{5}$
- C: `m = (a + b + c + d + e) / 5;`

Cebirsel İfadelerin C karşılığı

- Cebir: $m = \frac{a+b+c+d+e}{5}$
- C: `m = (a + b + c + d + e) / 5;`
- Cebir: $y = mx + b$

Cebirsel İfadelerin C karşılığı

- Cebir: $m = \frac{a+b+c+d+e}{5}$
- C: `m = (a + b + c + d + e) / 5;`
- Cebir: $y = mx + b$
- C: `y = m * x + b;`

Cebirsel İfadelerin C karşılığı

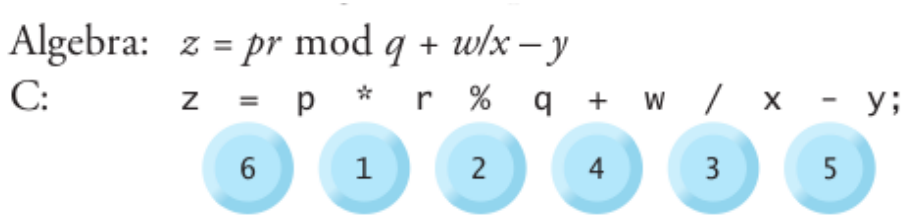
- Cebir: $m = \frac{a+b+c+d+e}{5}$
- C: `m = (a + b + c + d + e) / 5;`
- Cebir: $y = mx + b$
- C: `y = m * x + b;`
- Cebir: $z = pr \bmod q + w/x - y$

Cebirsel İfadelerin C karşılığı

- Cebir: $m = \frac{a+b+c+d+e}{5}$
- C: `m = (a + b + c + d + e) / 5;`
- Cebir: $y = mx + b$
- C: `y = m * x + b;`
- Cebir: $z = pr \bmod q + w/x - y$
- C: `z = p * r % q + w / x -y`

Cebirsel İfadelerin C karşılığı

- Cebir: $m = \frac{a+b+c+d+e}{5}$
- C: `m = (a + b + c + d + e) / 5;`
- Cebir: $y = mx + b$
- C: `y = m * x + b;`
- Cebir: $z = pr \bmod q + w/x - y$
- C: `z = p * r % q + w / x -y`



İkinci derece polinomun değerlendirilmesi

- `y = a * x * x + b * x + c;`

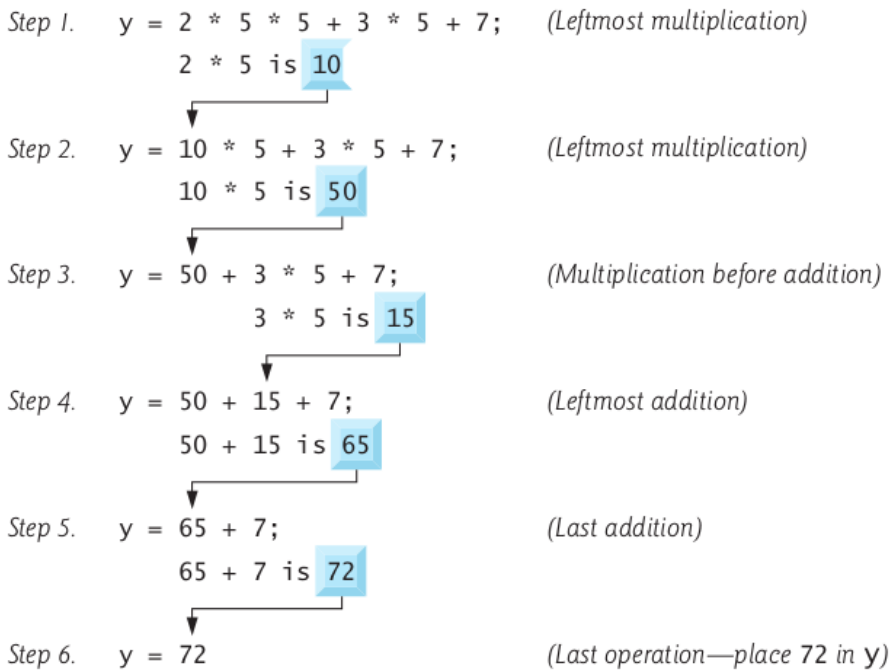
İkinci derece polinomun değerlendirilmesi

- `y = a * x * x + b * x + c;`



İkinci derece polinomun değerlendirilmesi

- $a = 2, b = 3, c = 7, x = 5$ olsun



Eşitlik ve İlişkisel Operatörler

Cebirsel eşitlik ya da ilişkisel operatör	C eşitlik ya da ilişkisel operatör	Örnek C koşulu	C koşulunun anlamı
İlişkisel operatörler			
$>$	`>`	`x > y`	x, y'den büyüktür
$<$	`<`	`x < y`	x, y'den küçüktür
$\geq$	`>=`	`x >= y`	x, y'den büyük veya eşittir
$\leq$	`<=`	`x <= y`	x, y'den küçük veya eşittir
Eşitlik operatörleri			
$=$	`==`	`x == y`	x, y'ye eşittir
$\neq$	`!=`	`x != y`	x, y'ye eşit değildir

- **Dikkat**: Karşılaştırma operatörü ==, atama operatörü = sıklıkla karıştırılabilir

Karşılaştırma Örneği

```c
// if komutunu, ilişkisel ve eşitlik
// operatörlerini kullanmak
#include <stdio.h>

// main fonksiyonu programın başlangıç noktasıdır
int main(void) {
    printf("İki tamsayı girin.\n");
    printf("Aralarındaki ilişkiyi size söyleyeceğim: ");

    int sayi1 = 0; // Kullanıcıdan alınacak ilk sayı
    int sayi2 = 0; // Kullanıcıdan alınacak ikinci sayı

    scanf("%d %d", &sayi1, &sayi2); // iki sayı oku

    if (sayi1 == sayi2) {
        printf("%d ve %d eşittir.\n", sayi1, sayi2);
    } // if sonu

    if (sayi1 != sayi2) {
        printf("%d ve %d eşit değildir.\n", sayi1, sayi2);
    } // if sonu
```

Karşılaştırma Örneği

```
22
23      if (sayi1 < sayi2) {
24          printf("%d, %d'den küçüktür.\n", sayi1, sayi2);
25      } // if sonu
26
27      if (sayi1 > sayi2) {
28          printf("%d, %d'den büyüktür.\n", sayi1, sayi2);
29      } // if sonu
30
31      if (sayi1 <= sayi2) {
32          printf("%d, %d'den küçük veya eşittir.\n", sayi1, sayi2);
33      } // if sonu
34
35      if (sayi1 >= sayi2) {
36          printf("%d, %d'den büyük veya eşittir.\n", sayi1, sayi2);
37      } // if sonu
38  } // main fonksiyonunun sonu
```

Karşılaştırma Örneği Çıktılar

```
İki tamsayı girin.
Aralarındaki ilişkiyi size söyleyeceğim: 3 7
3 ve 7 eşit değildir.
3, 7'den küçüktür.
3, 7'den küçük veya eşittir.
```

```
İki tamsayı girin.
Aralarındaki ilişkiyi size söyleyeceğim: 22 12
22 ve 12 eşit değildir.
22, 12'den büyüktür.
22, 12'den büyük veya eşittir.
```

```
İki tamsayı girin.
Aralarındaki ilişkiyi size söyleyeceğim: 20 20
20 ve 20 eşittir.
20, 20'den küçük veya eşittir.
20, 20'den büyük veya eşittir.
```

Sayıların Karşılaştırılması

```
15      if (sayi1 == sayi2) {
16          printf("%d ve %d eşittir.\n", sayi1, sayi2);
17      } // if sonu
```

- Üstteki ifade, `sayi1` ve `sayi2` değerlerini eşitlik bakımından karşılaştırır
- `if` ifadesinin gövdesi { ile başlar, } ile biter
- **Dikkat**: `if` ifadesindeki parantez kapatmanın sonuna ; sembolü yerleştirmek genellikle hatalı bir işlemdir
  - Yapılırsa koşulun etkisi olmayacaktır, gövde her durumda çalışacaktır

İşlem Önceliği

İşlem	Gruplama
( )	soldan sağa
* / %	soldan sağa
+ -	soldan sağa
< <= > >=	soldan sağa
== !=	soldan sağa
=	**sağdan sola**

Anahtar Sözcükler

Anahtar Sözcükler				
auto	do	goto	signed	unsigned
break	double	if	sizeof	void
case	else	int	static	volatile
char	enum	long	struct	while
const	extern	register	switch	
continue	float	return	typeof	
default	for	short	union	

- C99 standardında eklenen anahtar sözcükler:

  `_Bool _Complex _Imaginary inline strict`

- C11 standardında eklenen anahtar sözcükler:

  `_Alignas _Alignof _Atomic _Generic _Noreturn _Static_assert _Thread_local`

Alıştırmalar I

1. Kullanıcıdan alınan bir sayının tek sayı mı, çift sayı mı olduğunu yazdıran program
2. Kullanıcıdan alınan iki sayıdan ilki diğerinin katı mıdır, yazdıran program
3. Kullanıcıdan alınan beş basamaklı bir sayının basamaklarını alt alta yazdıran program
   - Kullanıcının beş basamaklı bir sayı gireceği varsayılacaktır, alınan sayının beş basamaklı olup olmadığının kontrolünü yapmanız istenmemektedir
4. Kullanıcıdan alınan üç değerden en küçüğü ve en büyüğünü ekrana yazdıran program
5. Yarıçap(r) değerini kullanıcıdan alarak, dairenin çevresini, alanını ve çapını yazdıran program
   - π sayısı tanımı için `double pi = 3.141592;` kullanılabilir
   - Yazdırma işlemi için `printf("%lf", pi);` kullanılabilir

Alıştırmalar II

6. Vücut kitle indeksi, ağırlığın(kg) boyun(m) karesine bölünmesi ile elde edilir($\frac{a}{b^2}$). Kullanıcıdan kilo ve boyu alan ve aşağıdaki değerlere göre kilo durumunu yazdıran program
   - 18.5'den küçük: Zayıf
   - 18.5 ile 24.99 arası: Normal kilolu
   - 25 ile 29.99 arası: Fazla kilolu
   - 30 üzeri: Obez
7. Kullanıcıdan üç adet sayı alan ve bu sayıları azalan sırada yazdıran program